STARVNT

VENDOR OS

MASTER BUG-FIX & STATE-TRUTH DIRECTIVE

Assigned Developer: Yogesh
Ownership: End-to-End
Priority: P0 Production Stability + Core State Integrity

ONE VENDOR → ONE CORE TRUTH → ALL SCREENS CONSISTENT

Execution & Acceptance Document

1. Scope & Objective

Current Vendor OS flow:

```
Login → Dashboard → Enquiries → Bookings → Calendar → Portfolio → Services → Availability → Payments → Reviews → Analytics → Messages → Profile/Hub → Aura+
```

> **Main problem is NOT UI polish. Main problem is STATE TRUTH.**

The same Vendor must not expose different business truth across Dashboard, Services, Profile, Availability, Payments, Reviews, Analytics and Aura+.

2. YOUR OWNERSHIP

This entire bug-fix scope has a single technical owner: Yogesh.

• Audit existing implementation.
• Find existing Core/API/DB logic before creating new logic.
• Identify duplicate business logic and duplicate sources of truth.
• Fix Core/API state.
• Verify DB mappings.
• Fix frontend state rendering.
• Ground Aura+ against Core.
• Verify security/authorization.
• Perform end-to-end QA.

> **Final technical ownership and verification = Yogesh.**

3. FIRST RULE — ONE SOURCE OF TRUTH

STARVNT Core must be authoritative for:

```
Vendor
↓
Profile
↓
Services
↓
Capability
↓
Capacity
↓
Coverage
↓
Availability
↓
Commercial Readiness
↓
Verification
↓
Matching Eligibility
↓
Lead Activation
↓
Opportunity
↓
Quote
↓
Payment
↓
Booking
↓
Execution
↓
Completion
↓
Settlement
↓
Review
↓
Performance
```

Frontend must not manufacture business state. Aura+ must not independently decide business state.

□ P0 — MUST FIX BEFORE ANY P1/P2 WORK

P0-01 — Profile Completion ≠ Matching Eligibility

```
profileCompletion
onboardingStatus
verificationStatus
availabilityStatus
commercialReadiness
matchingEligible
leadActivationStatus
```

Never use: profileCompletion === 100 → matching active.

Example:
Profile Setup 100% Complete
KYC Pending
Availability Configured
Matching Eligibility Locked
Lead Activation Inactive

Or:
Profile Setup 100% Complete
Verification Verified
Matching Eligibility Eligible
Lead Activation Active

P0-02 — Remove State Contradictions

Examples:
Setup Incomplete (100%)
5/5 Steps Done
100% Completion

Documents Verified
Next Step: KYC Verification

Matching Locked
while Aura+ says:
You are eligible for activation

One Core state → one API interpretation → one UI interpretation → one Aura+ explanation.

P0-03 — KYC State

```
NOT_SUBMITTED
SUBMITTED
UNDER_REVIEW
VERIFIED
REJECTED
EXPIRED
```

VERIFIED:
KYC Verified ✓
Documents verified by STARVNT
View Verification

UNDER_REVIEW:
KYC Verification Pending
Documents submitted — awaiting review

Do not show Upload Documents / Next step: KYC Verification after verification unless the vendor intentionally starts update/reverification.

P0-04 — Matching Eligibility

```
Minimum inputs:
Profile
Service
Category
Capability
Capacity
Coverage
Availability
Commercial Readiness
Verification

Output:
matchingEligible = true / false
```

If false, Core must provide structured reason codes:
KYC_PENDING
SERVICE_INCOMPLETE
CAPACITY_NOT_CONFIGURED
AVAILABILITY_NOT_CONFIGURED
COVERAGE_MISSING
COMMERCIAL_PROFILE_INCOMPLETE
CATEGORY_CONFIGURATION_INVALID

UI must show a human-readable reason.

P0-05 — Aura+ Grounding

Aura+ must never independently determine:
- Matching Eligibility
- Activation
- Availability
- Payment Success
- Booking Confirmation
- Settlement Eligibility
- Completion Verification

Flow:
Aura+ → Core API → Authoritative State → Aura+ Explanation

If Core says matchingEligible = false, Aura+ cannot say: “You are eligible.”

P0-06 — Category / Service Integrity

```
Current mismatch:
Service: S Royal Wedding Lawn & Ballroom
Category: Venue

Capabilities:
Cinematic
Drone 4K
Candid
Teaser Reel
```

Do not fix by changing labels only.

Separate:
Primary Category
Additional Category
Service
Capability
Capacity
Coverage
Availability

Pricing
Verification

Every additional category needs its own operational configuration.

P0-07 — Availability Engine

```
Never:
Working Hours = Operational Availability
```

Operational availability must consider:
Date, Time, Duration, Event Location, Service, Team, Equipment, Vehicle, Existing Booking, Hold, Travel, Setup, Teardown, Buffer, Concurrent Capacity.

Resource availability and vendor availability are different concepts.

P0-08 — Missing Resource Configuration

If category resources are required and No Operational Resources Declared, do not treat vendor as fully available.

Expected:
Capacity Not Configured
Matching Eligibility Pending

Exception only when Category Contract says resources are not required.

P0-09 — Overnight / Buffer

```
Availability must support:
Start
End
Overnight
Setup Buffer
Teardown Buffer
Travel Buffer
Breaks
```

Valid examples:
18:00 → 23:00
22:00 → 02:00

P0-10 — Portfolio / Media Truth

```
Current contradiction:
Media Library = 3
No Media Uploaded
```

Use one authoritative media source.
Project Count ≠ Media Count.
One project may contain multiple media.

If a filter returns zero:
“No media found for this filter.”

Not:
“Your media library is empty.”

P0-11 — Payment ≠ Settlement

```
Separate:
Customer Payment
Customer Balance
Payment Verification
Settlement Eligibility
```

```
Settlement Processing
Settled
```

Never blindly use Quote Total - Advance as vendor settlement. Apply the actual Commercial/Settlement Policy.

P0-12 — Rating Empty State

```
If verifiedReviewCount = 0
```

Show:
Rating = New
Would Recommend = —

Never show 5.0 + 0 reviews or 0.0 + 0 reviews + 0%.

P0-13 — Analytics Math

```
Never:
0 / 0 = 0%
```

Use — or “No data yet”. Analytics must use authoritative Core event data.

□ P1 — AFTER ALL P0 ARE VERIFIED

1. Complete Details → Review Profile when 100%.
2. Complete Your Profile → Review Your Profile.
3. Verified KYC → View Verification.
4. 1 active service(s) → 1 active service.
5. Configured ≠ Verified.
6. Published Portfolio ≠ Verified Portfolio.
7. Remove misleading +20% granted.
8. Project count ≠ media count.
9. Separate eventDate / createdAt / publishedAt / verifiedAt.
10. Use structured location.
11. Separate Category / Service / Event Type.
12. Add Availability to readiness.
13. Use structured equipment/resources.
14. Make crew capacity category-specific.
15. Make travel buffer policy-driven.
16. Define travel calculation: free radius, billable distance, one-way/round-trip, rate.
17. Define per-diem using crew count + days + rate.
18. Make accommodation policy-driven.
19. Verify “Core Escrow” terminology.
20. Separate Settlement Eligible / Processing / Settled.
21. Add strong payment references.
22. Use explicit payment states.
23. Make Google Maps connection state explicit.
24. Keep Google Reviews separate from STARVNT Reviews.
25. Use Verified Completed Bookings.
26. Rating must be only one ranking signal.

27. Keep Messages context-backed.

28. Messages + Enquiries must use the same Core Opportunity.

29. Enforce customer-data disclosure server-side.

30. Fix dead-end empty states.

□ P2 — AFTER P0/P1

• Secure Payments wording.

• Login authentication terminology.

• Email OTP / Mobile OTP / Google separation.

• Google account collision prevention.

• Vendor OS terminology standardization.

• Dynamic market-city claims.

• Remove unsupported trust metrics.

• Remove Live ₹0 style badges.

• Remove technical text such as 0 active in DB.

• Avoid absolute “all slots are open” messaging.

• Bookings → List / Calendar / Execution.

• Calendar feasibility.

• Calendar statuses.

• Analytics time filters.

• Turnaround metrics.

• Core funnel.

• YouTube/Vimeo validation.

• Blocked date reasons.

• Working-hour breaks.

• Remove placeholder website from DB.

• Structured city/hub.

• Aura shortcut navigation.

□ BACKEND / SECURITY VERIFICATION

Yogesh must verify:

V-01 Google OAuth account collision

V-02 Actual KYC status

V-03 Actual equipment/crew verification

V-04 matchingEligible

V-05 leadActivationStatus

V-06 Google Maps rating source

V-07 Login trust metrics source

V-08 Core Escrow implementation

V-09 Server-side payment verification

V-10 Payment webhook idempotency

V-11 Atomic booking/resource reservation

V-12 Cross-vendor data isolation

V-13 API authorization bypass

V-14 Analytics Core event source

V-15 Google rating separation

🔒 DEVELOPMENT ORDER — STRICT

PHASE 1 — CORE STATE

```
Profile
↓
Verification
↓
Service Readiness
↓
Capability
↓
Capacity
↓
Coverage
↓
Availability
↓
Commercial Readiness
↓
Matching Eligibility
↓
Lead Activation
```

PHASE 2 — BUSINESS FLOW

```
Opportunity
↓
Vendor Response
↓
Quote
↓
Payment Verification
↓
Booking
↓
Execution
↓
Completion Validation
↓
Settlement
↓
Review
```

PHASE 3 — UI

```
Dashboard
Services
Availability
Profile
Aura+
Payments
Reviews
Analytics
Messages
```

🔒 DEFINITION OF DONE

No issue may be marked DONE merely because the UI looks correct.

1. UI Proof — Correct state rendered.
2. API Proof — Correct authoritative API response.
3. DB Proof — Correct source record/state.
4. Business Logic Proof — Correct calculation/state transition.
5. Security Proof — Unauthorized access blocked.
6. E2E Proof — Vendor → Configuration → Eligibility → Opportunity → Quote → Payment → Booking → Execution → Completion → Settlement → Review.

CRITICAL RULE

UI 100% ≠ System 100%.

If Core State ≠ API State ≠ DB State ≠ Business Logic ≠ UI State:

STATUS = NOT COMPLETE

□ FINAL INSTRUCTION TO YOGESH

The goal of this sprint is not to make every screen look polished. The goal is:

ONE VENDOR → ONE CORE TRUTH → ALL SCREENS CONSISTENT

Dashboard □□ □□□□, Services □□□□□ □□□□.
Profile □□□□□ □□□□.
Availability □□□□□ □□□□.
Payments □□□□□ □□□□.
Reviews □□□□□ □□□□.
Analytics □□□□□ □□□□.
Aura+ □□□□□ explain □□□□.

CORE DECIDES → API DELIVERS → UI DISPLAYS → AURA+ EXPLAINS

First fix State Architecture. Then Business Logic. Then API/DB. Then UI.

No hardcoded business truth. No duplicate source of truth. No frontend-generated eligibility. No Aura+ invented state.

⚠ Do not mark the work complete until P0 + required backend/security verification + E2E tests pass.

Single owner: Yogesh.
Final accountability: Yogesh.